

PATRICIA MBOMA KASHWEKA

COMP# 2021527433

Database Design Write-up .The database design for the flight booking service comprises five main tables: **FLIGHTS**, **CARRIERS**, **MONTHS**, **WEEKDAYS** and **USERS**

FLIGHTS: This table includes `fid` (flight ID), `carrier_id`, `month_id`, `day_of_month`, `day_of_week_id`, `departure_delay`, `arrival_delay`, `actual_time`, `distance`, and `price`.

- Justification: It captures comprehensive flight details crucial for user searches and bookings.

CARRIERS: Contains `cid` (carrier ID) and `name` (carrier name).

- Justification: Separating carrier data promotes normalization and simplifies flight-related queries.

MONTHS: Stores `mid` (month ID) and `month` (month name).

- Justification: Allows for easy month referencing in the **FLIGHTS** table, enhancing data organization.

WEEKDAYS: Includes `did` (day ID) and `day_of_week` (day name).

- Justification: Facilitates weekday-based queries without redundancy.

USERS: Stores user account details, including `username`, `password_hash`, and `balance`.

- Justification: This information must be persistent for effective user authentication and transaction management.

RESERVATIONS: Manages the records of user bookings, linking to itineraries and payment status.

- Justification: Essential for tracking user reservations and ensuring smooth flight management.

INTINERARIES: Manages user travel plans by aggregating multiple flight IDs into a single entry.

Justification: Allows users to book direct or one-hop flights as a unified itinerary, thus simplifying the booking process and enhancing user experience.

Thought Process

In designing the database, I focused on normalization to avoid redundancy and ensure data integrity. The tables chosen for persistence are vital for maintaining consistent and retrievable information necessary for user interactions. Conversely, transient data, such as the currently logged-in user's session or temporary search results, is handled in-memory. This choice minimizes database load and optimizes performance, as such data does not require long-term storage. Overall, this structure supports application functionality while promoting efficient data management.

